

Conceptual Metamodel



Juan Manuel Cerón Certuche

juanmanuelceron@unicauca.edu.co

Harold Andrés Molano

hmolano@unicauca.edu.co

Felipe Armand Pino Sierra

fpinos@unicauca.edu.co

Yuliana Trujillo Vidal

Yatrujillo@unicauca.edu.co

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Content

1. Scope	4
2. Background	4
2.1 Basic Concepts	4
2.2 Example: Library System	5
3. Conformance	5
4. Normative References	5
5. Terms and Definitions	6
6. Symbols (Notation)	7
7. Additional Information	7
7.1. Conventions	7
8. Core Classes	8
8.1. Abstract Metaclass NamedElement	8
8.2. Metaclass Domain	8
8.3. Metaclass SubDomain	8
8.4. Metaclass Concept	9
8.5. Metaclass ValueObject	9
8.6. Metaclass Attribute	9
8.7. Metaclass Relationship	10
8.8. Metaclass BusinessService	10
8.9. Metaclass BusinessOperation	11
8.10. Metaclass Parameter	11
9. Example: Library Management	11
9.1. Instance Simple Model in XMI	12
9.2. Instance Complex Model in XMI	13
9.3. Mapping to Metaclasses	16
10. Well-Formedness Constraints	17
References	19

Abstract

English

This specification defines a metamodel for representing conceptual domain models oriented to the hexagonal architecture, including domains, subdomains, concepts, value objects, relationships between concepts, and business services with their operations. The metamodel is designed for model-driven engineering tasks such as validation (EVL), transformation (ETL), and code generation (EGL)..

Spanish

Esta especificación define un metamodelo para representar modelos conceptuales de dominio orientados a la arquitectura hexagonal, incluyendo dominios, subdominios, conceptos, objetos de valor, relaciones entre conceptos y servicios de negocio con sus operaciones. El metamodelo está diseñado para tareas de ingeniería dirigida por modelos como validación (EVL), transformación (ETL) y generación de código (EGL).

French

Cette spécification définit un métamodèle pour la représentation de modèles de domaine conceptuels orientés selon une architecture hexagonale, incluant les domaines, les sous-domaines, les concepts, les objets de valeur, les relations entre les concepts et les services métier ainsi que leurs opérations. Ce métamodèle est conçu pour les tâches d'ingénierie dirigée par les modèles telles que la validation (EVL), la transformation (ETL) et la génération de code (EGL).

1. Scope

This specification defines a metamodel designed to capture the conceptual knowledge of a software system from a business perspective, allowing for the structured representation of the fundamental elements of a domain and their interactions. The metamodel includes:

- Decomposition of the business into domains and subdomains as the primary organizational unit.
- Modeling of domain entities through Concepts, and of data without inherent identity through Value Objects.
- Definition of semantic relationships between Concepts, with explicit typing that distinguishes composition, aggregation, and inheritance.
- Specification of Business Services by grouping operations with input parameters and return types.
- Description of typed attributes, indicating their mandatory nature within Concepts and Value Objects.

The purpose of this metamodel is to serve as a starting point for automated model-driven engineering processes. It does not address implementation aspects such as persistence, security, domain event handling, or communication between services, which are the responsibility of the architectural layers derived from it.

2. Background

Domain-driven design (DDD) is a software development approach that centers the modeling process on a deep understanding of the business, promoting close collaboration between domain experts and technical teams [1]. This approach is especially valuable in complex systems where business logic is at the core of the system, and it has been widely adopted in modern architectures such as hexagonal, microservices, and event-driven architectures [2].

2.1 Basic Concepts

The following concepts are standard in the literature [1]:

- **Domain:** The problem space that the system addresses. It represents the area of knowledge or activity around which the software solution revolves. Example: Library, E-commerce.
- **Subdomain:** A cohesive partition within a domain that groups related concepts. Example: Loan Management, Book Catalog.
- **Concept:** An entity within the domain with its own identity that persists over time. Example: Book, User, Loan.
- **Value Object:** An element within the domain that is defined solely by its attributes, without its own identity. Example: Address, Date Range.
- **Relationship:** A directed and typed association between two Concepts. It can be composition, aggregation, or inheritance, depending on the business semantics.
- **Business Service:** A component that encapsulates domain operations that

do not naturally belong to any Concept. Example: PreferenceService, LoanManager.

- Business Operation: A specific action within a Business Service, with input parameters and a defined return type.

2.2 Example: Library System

A representative example is a library management system. The main domain is LibrarySystem, which contains a subdomain called LoanManagement, dedicated to managing book loans. Within this subdomain, three Concepts are defined: Book, which groups attributes such as isbn, title, author, publicationYear, and status; User, which describes library members using userId, name, email, and membershipDate; and Loan, which represents the act of borrowing with the attributes loanId, loanDate, dueDate, and returnDate. The relationships between concepts are association-based: loan_references_book links Book to Loan, and loan_references_user links User to Loan, thus modeling that a loan references both the book and the user involved.

The subdomain also defines a Business Service called LoanService, which operates on the three concepts above and exposes three operations: borrowBook(userId: String, isbn: String): Loan, which registers a new loan; `returnBook(loanId: String)` : A Boolean that processes the return; and `checkOverdue(referenceDate: LocalDate)` : A List<Loan> that identifies overdue loans as of a reference date.

This instance model conforms to the metamodel defined in this document and is used throughout to illustrate fundamental concepts, as well as to serve as a test model in validation (EVL), transformation (ETL), and code generation (EGL) tasks.

3. Conformance

An implementation complies with this specification if it supports the creation, storage, and exchange of models that adhere to the metaclasses, attributes, and relationships defined in the metamodel, and if it respects the structural and semantic integrity constraints described in the EVL validation module. Specifically, any tool capable of correctly instantiating a domain with its subdomains, concepts, value objects, relationships, business services, and operations, following the containment hierarchy established in the metamodel, is considered compliant.

4. Normative References

- OMG MOF 2.5.1 – Meta-Object Facility Core Specification. Conceptual basis for defining the metamodel.
- Eclipse EMF – Eclipse Modeling Framework. Reference platform for implementing the metamodel in Ecore format.

- Epsilon ETL – Epsilon Transformation Language. Language used to transform the conceptual model into the hexagonal architecture model.
- Epsilon EVL – Epsilon Validation Language. Language used to validate constraints on metamodel instances.
- Epsilon EGL/EGX – Epsilon Generation Language. Language used to generate PlantUML diagrams from the conceptual model.
- RFC 2119 – Key words for use in RFCs to Indicate Requirement Levels. Convention used for the normative terms employed in this specification.

5. Terms and Definitions

Term	Definition
Domain	A model that describes the conceptual structure of a software system in terms of business knowledge, grouping subdomains, concepts, relationships, and services.
Subdomain	A cohesive partition within a domain that groups conceptually related elements, such as concepts, value objects, relationships, and business services.
Concept	An abstraction of a domain entity with its own identity that persists over time and possesses attributes that characterize it.
ValueObject	An element of the domain defined solely by the value of its attributes, without its own identity. It is immutable and belongs to a subdomain.
Relationship	A directed and typed association between two Concepts within the same subdomain. It can be of the composition, aggregation, or inheritance type.
BusinessService	A component that encapsulates a set of business operations that do not naturally belong to any Concept, and that operates on one or more concepts of the subdomain.
BusinessOperation	A specific action exposed by a Business Service, with a defined return type and a set of input parameters.
Attribute	A typed property that characterizes a Concept or a Value Object. It can be marked as mandatory or optional.
Parameter	An input data item associated with a Business Operation, with a name, a type, and an indication of whether it is required or not.

6. Symbols (Notation)

In the diagrams that accompany this specification, the following notation is used:

- Boxes represent metaclasses.
- Lines with solid diamonds represent composition (containment).
- Lines with simple arrows represent non-containment references.
- Attributes are shown inside the box.

7. Additional Information

7.1. Conventions

The keywords MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in RFC 2119.

5

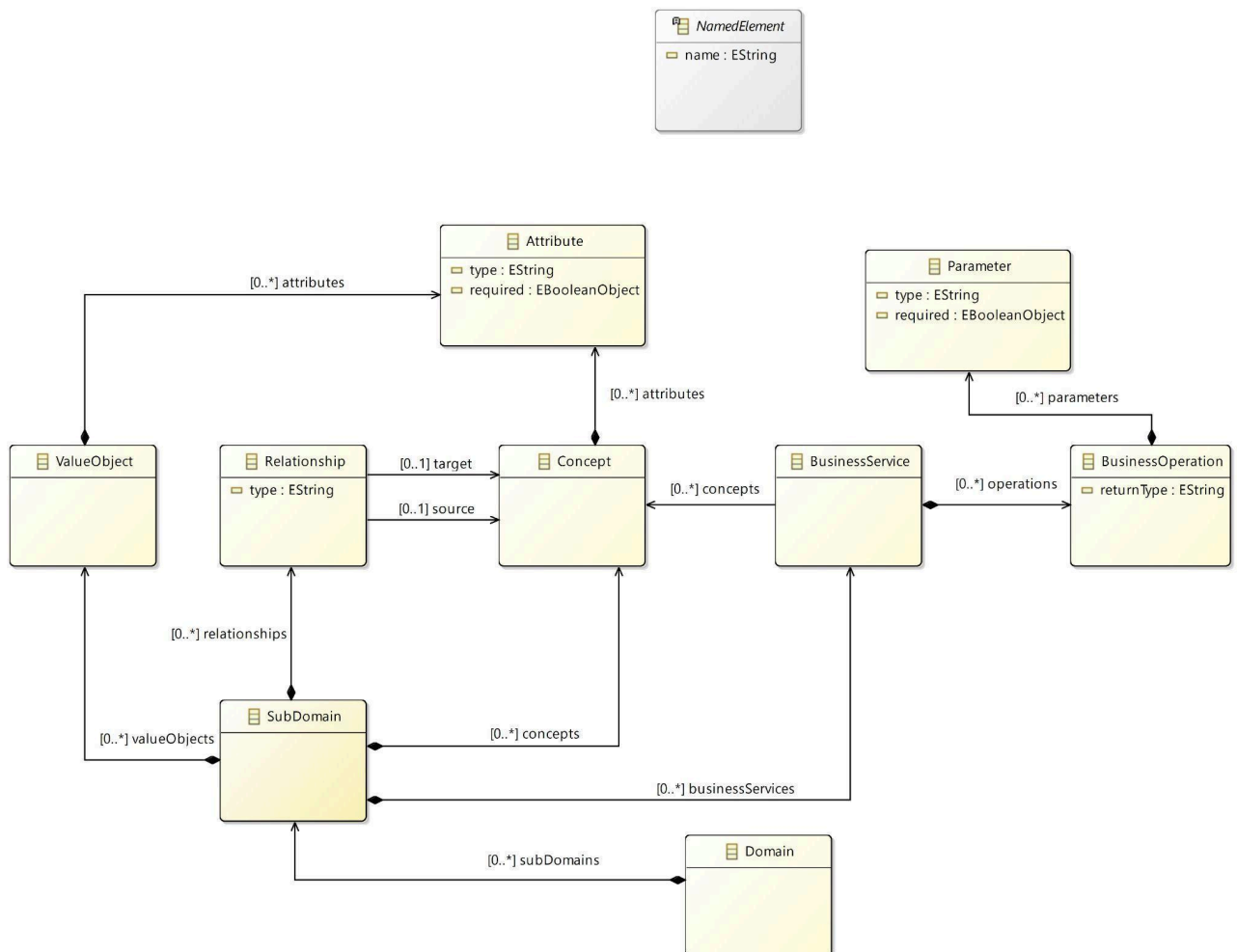


Figure 1. Metamodel

8. Core Classes

This section specifies the metaclasses that constitute the Conceptual metamodel. The metamodel is defined using the Eclipse Modeling Framework (EMF) and follows the Ecore syntax. As shown in Figure 1, the main concepts are: a Domain contains one or more SubDomains; each SubDomain groups a set of Concepts, ValueObjects, Relationships, and BusinessServices. Concepts and ValueObjects are domain elements that have Attributes as contained elements. Relationships connect a source Concept to a target Concept and carry a type attribute that defines their semantics. BusinessServices group a set of BusinessOperations, each with its input Parameters and a return type. All metamodel elements inherit from the abstract metaclass NamedElement, which provides the common name attribute. The following subsections describe each metaclass in detail, including its attributes, references, and well-formed constraints.

8.1. Abstract Metaclass NamedElement

Abstract metaclass, root of the hierarchy. Provides the common name attribute to all elements of the metamodel.

Superclass: none (abstract root metaclass).

Attributes:

Name	Type	Multiplicity	Description
<i>name</i>	EString	1	Model Element Name

8.2. Metaclass Domain

Root element of the conceptual model. Represents the system's problem space.

Superclass: NamedElement.

Attributes: none (inherits name).

References:

Name	Type	Opposite	Aggregation	Description
subDomains	SubDomain	(none)	Containment	The subdomains that make up this domain

8.3. Metaclass SubDomain

A cohesive partition within a domain that groups related conceptual elements.

Superclass: NamedElement.

Attributes: none (inherits name).

References:

Name	Type	Opposite	Aggregation	Description
------	------	----------	-------------	-------------

concepts	Concept	(none)	Containment	Subdomain concepts
valueObjects	ValueObject	(none)	Containment	The value objects of the subdomain
relationships	Relationship	(none)	Containment	The relationships between concepts in the subdomain
businessServices	BusinessService	(none)	Containment	The business services of the subdomain

8.4. Metaclass Concept

It represents a domain entity with its own identity that persists over time.

Superclass: NamedElement.

Attributes: none (inherits name).

References:

Name	Type	Opposite	Aggregation	Description
attributes	Attribute	(none)	Containment	The attributes that characterize the concept

8.5. Metaclass ValueObject

It represents an element of the domain defined solely by its attributes, without its own identity.

Superclass: NamedElement.

Attributes: none (inherits name).

References:

Name	Type	Opposite	Aggregation	Description
attributes	Attribute	(none)	Containment	The attributes that define value object

8.6. Metaclass Attribute

A typed property that characterizes a Concept or ValueObject.

Superclass: NamedElement

Attributes:

Name	Type	Multiplicity	Description
type	EString	1	Attribute data type
required	EBooleanObject	0..1	Indicates whether the attribute is required

8.7. Metaclass Relationship

Directed and typed association between two Concepts within the same subdomain.

Superclass: NamedElement.

Attributes:

Name	Type	Multiplicity	Description
type	EString	1	Relation type: composition, aggregation or inheritance.

References:

Name	Type	Opposite	Aggregation	Description
Source	Concept	(none)	None	Source concept of the relationship
target	Concept	(none)	None	Target concept of the relationship

8.8. Metaclass BusinessService

Directed and typed association between two Concepts within the same subdomain.

Superclass: NamedElement.

A component that encapsulates business operations that don't naturally belong to any Concept.

Superclass: NamedElement.

Attributes: none (inherits name).

References:

Name	Type	Opposite	Aggregation	Description
operations	BusinessOperation	(none)	Containment	The operations exposed by the service
concepts	Concept	(none)	None	The concepts on

				which the service operates
--	--	--	--	----------------------------

8.9. Metaclass BusinessOperation

Specific action exposed by a BusinessService, with input parameters and return type.

Superclass: NamedElement

Attributes:

Name	Type	Multiplicity	Description
returnType	EString	1	Data type returned by the operation

References:

Name	Type	Opposite	Aggregation	Description
parameters	Parameter	(none)	Containment	The operation's input parameters

8.10. Metaclass Parameter

Input data associated with a BusinessOperation.

Superclass: NamedElement.

Attributes:

Name	Type	Multiplicity	Description
type	EString	1	Parameter data type
required	EBooleanObject	0..1	Indicates whether the parameter is required

9. Example: Library Management

The following instance model represents a library management system with a main domain, a loan management subdomain, three concepts, two relationships, and a business service with three operations. The root domain is LibrarySystem, which contains the LoanManagement subdomain. Within this subdomain, the concepts Book, User, and Loan are defined, each with its respective typed attributes marked according to their mandatory nature. The relationships lend_references_book and lend_references_user connect Book and User to Loan, respectively,

both of type association. The business service `LoanService` operates on the three concepts and exposes the operations `lendBook`, `returnBook`, and `checkOverdue`, each with its defined input parameters and return type. This instance model is used throughout the document as a test model for validation (EVL), transformation (ETL), and code generation (EGL) tasks, and its XMI representation is shown below:

9.1. Instance Simple Model in XMI

```
<?xml version="1.0" encoding="UTF-8"?>
<conceptual:Domain
  xmi:version="2.0"
  xmlns:xmi="http://www.omg.org/XMI"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:conceptual="http://www.hexagen.org/conceptual"
  xsi:schemaLocation="http://www.hexagen.org/conceptual hexagenModel.ecore"
  name="LibrarySystem">

  <!-- Subdominio: Gestión de Préstamos -->
  <subDomains name="LoanManagement">

    <!-- Concept: Book -->
    <concepts name="Book">
      <attributes name="isbn" type="String" required="true"/>
      <attributes name="title" type="String" required="true"/>
      <attributes name="author" type="String" required="true"/>
      <attributes name="publicationYear" type="Integer" required="false"/>
      <attributes name="status" type="BookStatus" required="true"/>
    </concepts>

    <!-- Concept: User -->
    <concepts name="User">
      <attributes name="userId" type="String" required="true"/>
      <attributes name="name" type="String" required="true"/>
      <attributes name="email" type="String" required="true"/>
      <attributes name="membershipDate" type="LocalDate" required="true"/>
    </concepts>

    <!-- Concept: Loan -->
    <concepts name="Loan">
      <attributes name="loanId" type="String" required="true"/>
      <attributes name="loanDate" type="LocalDate" required="true"/>
      <attributes name="dueDate" type="LocalDate" required="true"/>
      <attributes name="returnDate" type="LocalDate" required="false"/>
    </concepts>

    <!-- Relationships -->
    <relationships name="loan_references_book" type="association">
      <source href="#/subDomains.0/concepts.0"/>
      <target href="#/subDomains.0/concepts.2"/>
    </relationships>

    <relationships name="loan_references_user" type="association">
      <source href="#/subDomains.0/concepts.1"/>
      <target href="#/subDomains.0/concepts.2"/>
    </relationships>

    <!-- BusinessService: LoanService -->
    <businessServices name="LoanService">
      <operations name="borrowBook" returnType="Loan">
        <parameters name="userId" type="String" required="true"/>
        <parameters name="isbn" type="String" required="true"/>
      </operations>
      <operations name="returnBook" returnType="Boolean">
        <parameters name="loanId" type="String" required="true"/>
      </operations>
      <operations name="checkOverdue" returnType="List<Loan>*>">
        <parameters name="referenceDate" type="LocalDate" required="true"/>
      </operations>
    </businessServices href="#/subDomains.0/concepts.0"/>
  </subDomains>
</conceptual:Domain>
```

```

<concepts href="#/subDomains.0/concepts.1"/>
<concepts href="#/subDomains.0/concepts.2"/>
</businessServices>

```

```

</subDomains>

```

```

</conceptual:Domain>

```

9.2. Instance Complex Model in XMI

```

<?xml version="1.0" encoding="UTF-8"?>
<conceptual:Domain
xmi:version="2.0"
xmlns:xmi="http://www.omg.org/XMI"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:conceptual="http://www.hexagen.org/conceptual"
xsi:schemaLocation="http://www.hexagen.org/conceptual hexagenModel.ecore"
name="LibrarySystem">

```

```

<!-- =====
SUBDOMINIO 1: Gestión de Catálogo
===== -->

```

```

<subDomains name="CatalogManagement">

```

```

<!-- Concepts -->
<concepts name="Book">
<attributes name="isbn" type="String" required="true"/>
<attributes name="title" type="String" required="true"/>
<attributes name="author" type="String" required="true"/>
<attributes name="publicationYear" type="Integer" required="false"/>
<attributes name="status" type="BookStatus" required="true"/>
<attributes name="totalCopies" type="Integer" required="true"/>
<attributes name="availableCopies" type="Integer" required="true"/>
</concepts>

```

```

<concepts name="Author">
<attributes name="authorId" type="String" required="true"/>
<attributes name="fullName" type="String" required="true"/>
<attributes name="birthDate" type="LocalDate" required="false"/>
<attributes name="nationality" type="String" required="false"/>
</concepts>

```

```

<concepts name="Category">
<attributes name="categoryId" type="String" required="true"/>
<attributes name="name" type="String" required="true"/>
<attributes name="description" type="String" required="false"/>
</concepts>

```

```

<!-- ValueObjects -->
<valueObjects name="BookIdentifier">
<attributes name="isbn" type="String" required="true"/>
<attributes name="edition" type="String" required="false"/>
</valueObjects>

```

```

<valueObjects name="SearchCriteria">
<attributes name="keyword" type="String" required="false"/>
<attributes name="category" type="String" required="false"/>
<attributes name="yearRange" type="String" required="false"/>
</valueObjects>

```

```

<!-- Relationships -->
<relationships name="book_written_by_author" type="association">
<source href="#/subDomains.0/concepts.0"/>
<target href="#/subDomains.0/concepts.1"/>
</relationships>

```

```

<relationships name="book_belongs_to_category" type="association">
<source href="#/subDomains.0/concepts.0"/>
<target href="#/subDomains.0/concepts.2"/>
</relationships>

```

```

<!-- BusinessServices -->

```

```

<businessServices name="CatalogService">
  <operations name="searchBooks" returnType="List<Book>">
    <parameters name="criteria" type="SearchCriteria" required="true"/>
  </operations>
  <operations name="addBook" returnType="Book">
    <parameters name="isbn" type="String" required="true"/>
    <parameters name="title" type="String" required="true"/>
    <parameters name="authorId" type="String" required="true"/>
  </operations>
  <operations name="updateBook" returnType="Boolean">
    <parameters name="isbn" type="String" required="true"/>
    <parameters name="updates" type="Map<String, Object>" required="true"/>
  </operations>
  <operations name="removeBook" returnType="Boolean">
    <parameters name="isbn" type="String" required="true"/>
  </operations>
  <concepts href="#/subDomains.0/concepts.0"/>
  <concepts href="#/subDomains.0/concepts.1"/>
  <concepts href="#/subDomains.0/concepts.2"/>
</businessServices>

</subDomains>

<!-- =====
SUBDOMINIO 2: Gestión de Usuarios y Préstamos
===== -->
<subDomains name="LoanManagement">

  <!-- Concepts -->
  <concepts name="User">
    <attributes name="userId" type="String" required="true"/>
    <attributes name="name" type="String" required="true"/>
    <attributes name="email" type="String" required="true"/>
    <attributes name="membershipDate" type="LocalDate" required="true"/>
    <attributes name="status" type="UserStatus" required="true"/>
    <attributes name="borrowedCount" type="Integer" required="true"/>
  </concepts>

  <concepts name="Loan">
    <attributes name="loanId" type="String" required="true"/>
    <attributes name="loanDate" type="LocalDate" required="true"/>
    <attributes name="dueDate" type="LocalDate" required="true"/>
    <attributes name="returnDate" type="LocalDate" required="false"/>
    <attributes name="fineAmount" type="BigDecimal" required="false"/>
  </concepts>

  <concepts name="Reservation">
    <attributes name="reservationId" type="String" required="true"/>
    <attributes name="reservationDate" type="LocalDate" required="true"/>
    <attributes name="expiryDate" type="LocalDate" required="true"/>
    <attributes name="status" type="ReservationStatus" required="true"/>
  </concepts>

  <!-- ValueObjects -->
  <valueObjects name="Money">
    <attributes name="amount" type="BigDecimal" required="true"/>
    <attributes name="currency" type="String" required="true"/>
  </valueObjects>

  <valueObjects name="Notification">
    <attributes name="channel" type="NotificationChannel" required="true"/>
    <attributes name="recipient" type="String" required="true"/>
    <attributes name="message" type="String" required="true"/>
  </valueObjects>

  <!-- Relationships -->
  <relationships name="loan_borrowed_by_user" type="composition">
    <source href="#/subDomains.1/concepts.0"/>
    <target href="#/subDomains.1/concepts.1"/>
  </relationships>

  <relationships name="loan_references_book" type="association">
    <source href="#/subDomains.0/concepts.0"/>
    <target href="#/subDomains.1/concepts.1"/>

```

```

</relationships>

<relationships name="reservation_made_by_user" type="composition">
<source href="#/subDomains.1/concepts.0"/>
<target href="#/subDomains.1/concepts.2"/>
</relationships>

<relationships name="reservation_for_book" type="association">
<source href="#/subDomains.0/concepts.0"/>
<target href="#/subDomains.1/concepts.2"/>
</relationships>

<!-- BusinessServices -->
<businessServices name="LoanService">
<operations name="borrowBook" returnType="Loan">
<parameters name="userId" type="String" required="true"/>
<parameters name="isbn" type="String" required="true"/>
</operations>
<operations name="returnBook" returnType="Money">
<parameters name="loanId" type="String" required="true"/>
</operations>
<operations name="renewLoan" returnType="Boolean">
<parameters name="loanId" type="String" required="true"/>
<parameters name="extensionDays" type="Integer" required="true"/>
</operations>
<operations name="checkOverdue" returnType="List<Loan>";>
<parameters name="referenceDate" type="LocalDate" required="true"/>
</operations>
<concepts href="#/subDomains.1/concepts.0"/>
<concepts href="#/subDomains.1/concepts.1"/>
<concepts href="#/subDomains.0/concepts.0"/>
</businessServices>

<businessServices name="UserService">
<operations name="registerUser" returnType="User">
<parameters name="name" type="String" required="true"/>
<parameters name="email" type="String" required="true"/>
</operations>
<operations name="updateUser" returnType="Boolean">
<parameters name="userId" type="String" required="true"/>
<parameters name="updates" type="Map<String, Object>"; required="true"/>
</operations>
<operations name="suspendUser" returnType="Boolean">
<parameters name="userId" type="String" required="true"/>
<parameters name="reason" type="String" required="true"/>
</operations>
<concepts href="#/subDomains.1/concepts.0"/>
</businessServices>

<businessServices name="ReservationService">
<operations name="createReservation" returnType="Reservation">
<parameters name="userId" type="String" required="true"/>
<parameters name="isbn" type="String" required="true"/>
</operations>
<operations name="cancelReservation" returnType="Boolean">
<parameters name="reservationId" type="String" required="true"/>
</operations>
<operations name="notifyAvailability" returnType="Boolean">
<parameters name="reservationId" type="String" required="true"/>
<parameters name="notification" type="Notification" required="true"/>
</operations>
<concepts href="#/subDomains.1/concepts.0"/>
<concepts href="#/subDomains.1/concepts.2"/>
<concepts href="#/subDomains.0/concepts.0"/>
</businessServices>

</subDomains>

<!-- =====
SUBDOMINIO 3: Gestión de Multas
===== -->
<subDomains name="FineManagement">

<concepts name="Fine">

```

```

<attributes name="fineId" type="String" required="true"/>
<attributes name="amount" type="BigDecimal" required="true"/>
<attributes name="issueDate" type="LocalDate" required="true"/>
<attributes name="paidDate" type="LocalDate" required="false"/>
<attributes name="status" type="FineStatus" required="true"/>
</concepts>

<valueObjects name="FineCalculation">
<attributes name="daysOverdue" type="Integer" required="true"/>
<attributes name="dailyRate" type="BigDecimal" required="true"/>
<attributes name="total" type="BigDecimal" required="true"/>
</valueObjects>

<relationships name="fine_applied_to_loan" type="association">
<source href="#/subDomains.1/concepts.1"/>
<target href="#/subDomains.2/concepts.0"/>
</relationships>

<businessServices name="FineService">
<operations name="calculateFine" returnType="FineCalculation">
<parameters name="dueDate" type="LocalDate" required="true"/>
<parameters name="returnDate" type="LocalDate" required="true"/>
</operations>
<operations name="issueFine" returnType="Fine">
<parameters name="loanId" type="String" required="true"/>
<parameters name="amount" type="BigDecimal" required="true"/>
</operations>
<concepts href="#/subDomains.2/concepts.0"/>
<concepts href="#/subDomains.1/concepts.1"/>
</businessServices>

</subDomains>

</conceptual:Domain>

```

9.3. Mapping to Metaclasses

XMI Element	Metaclass	Explanation
<conceptual:Domain name="LibrarySystem">	Domain	Root container of the model
<subDomains name="LoanManagement">	SubDomain	Single subdomain of the system
<concepts name="Book">	Concept	Book entity with its attributes
<concepts name="User">	Concept	User entity with its attributes
<concepts name="Loan">	Concept	Loan entity with its attributes
<attributes name="isbn">	Attribute	Mandatory attribute of

type="String" required="true"/>		Book
<attributes name="returnDate" type="LocalDate" required="false" >	Attribute	Optional attribute of Loan
<relationships name="loan_references_book" type="association">	Relationship	Association from Book to Loan
<relationships name="loan_references_user" type="association">	Relationship	Association from User to Loan
<businessServices name="LoanService">	BusinessService	Service operating over the three concepts
<operations name="borrowBook" returnType="Loan">	BusinessOperation	Operation to register a loan
<operations name="returnBook" returnType="Boolean">	BusinessOperation	Operation to process a book return
<operations name="checkOverdue" returnType="List<Loan>">	BusinessOperation	Operation to query overdue loans
<parameters name="userId" type="String" required="true"/>	Parameter	Mandatory parameter of borrowBook
<concepts href="#//subDo	Reference	LoanService references Book

mains.0/concept s.0"/>		
---------------------------	--	--

10. Well-Formedness Constraints

The following informal constraints can be formalised in EVL (Epsilon Validation Language) for model validation.

NamedElementHasName – Every element in the model must have a non-empty name.

DomainHasAtLeastOneSubDomain – A Domain must contain at least one SubDomain.

SubDomainNamesUniqueWithinDomain – Within a Domain, all SubDomain names must be unique.

SubDomainHasAtLeastOneConcept – A SubDomain must contain at least one Concept.

ConceptNamesUniqueWithinSubDomain – Within a SubDomain, all Concept names must be unique.

ValueObjectNamesUniqueWithinSubDomain – Within a SubDomain, all ValueObject names must be unique.

BusinessServiceNamesUniqueWithinSubDomain – Within a SubDomain, all BusinessService names must be unique.

RelationshipSourceAndTargetDefined – Every Relationship must have both a source and a target Concept defined.

RelationshipSourceAndTargetWithinSubDomain – The source and target Concepts of a Relationship must belong to the same SubDomain.

RelationshipTypeValid – The type of a Relationship must be one of the accepted values: composition, aggregation, or inheritance.

BusinessServiceHasAtLeastOneOperation – A BusinessService must expose at least one BusinessOperation.

BusinessOperationHasReturnType – Every BusinessOperation must have a non-empty return type.

AttributeHasType – Every Attribute must have a non-empty type defined.

ParameterHasType – Every Parameter must have a non-empty type defined.

BusinessServiceReferencesExistingConcepts – Every Concept referenced by a BusinessService must exist within the same SubDomain.

References

- [1] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- [2] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
- [3] OMG. (2017). *Unified Modeling Language (UML) Specification*, Version 2.5.1. Object Management Group. formal/17-12-05.